

Documentação do Projeto

Servidor de Armazenamento Local com Acesso Remoto e Automação

1. Sobre o Projeto

A ideia deste projeto nasceu de uma necessidade bem comum: transformar um computador normal em um servidor particular de armazenamento de arquivos. O objetivo é utilizar o espaço de disco desse computador para guardar documentos, fotos e outros arquivos, permitindo que eles sejam acessados, baixados ou alterados através de uma página na internet.

O coração do sistema foi construído em Python, usando uma ferramenta chamada Flask, que cria a interface web onde o usuário navega nas pastas. Como os arquivos ficam salvos fisicamente no computador, ele atua como o nosso "servidor".

Para resolver o desafio de acessar essa máquina de qualquer lugar do mundo — sem precisar de configurações avançadas no roteador — integramos o **Cloudflare Tunnel**. Por fim, para que o sistema não dependa de alguém ligando tudo manualmente, criamos uma automação em PowerShell configurada como um serviço do Windows. Assim, quando o computador é ligado, o servidor web sobe, o túnel é aberto, e um script Python envia um e-mail automaticamente com o link de acesso daquele dia.

2. Tecnologias Utilizadas (Parte 1)

Para construir a solução, juntei várias tecnologias. Vou explicar o que cada uma faz de forma bem direta:

- **Python:** É a linguagem de programação principal do projeto. Ela foi escolhida por ser simples, direta e ter ótimas ferramentas para lidar com arquivos do sistema e criar servidores web de forma rápida.
- **Flask:** É um "micro-framework" para Python. Basicamente, é uma caixa de ferramentas que ajuda o Python a entender requisições de internet (quando alguém digita um site) e devolver páginas web. Ele é o motor por trás do nosso site de arquivos.
- **HTML, CSS e JavaScript:** São o trio tradicional da internet. O HTML constrói a estrutura da página de arquivos (botões, listas de pastas), o CSS cuida da beleza (cores, tamanhos) e o JavaScript ajuda nas interações da tela sem precisar recarregar a página o tempo todo.
- **PowerShell:** É o terminal avançado do Windows, usado para criar scripts que controlam o sistema operacional. Neste projeto, ele é o maestro da orquestra: criamos um script que liga o Flask, liga o Cloudflare e chama o envio de e-mail na ordem certa.

2. Tecnologias Utilizadas (Parte 2)

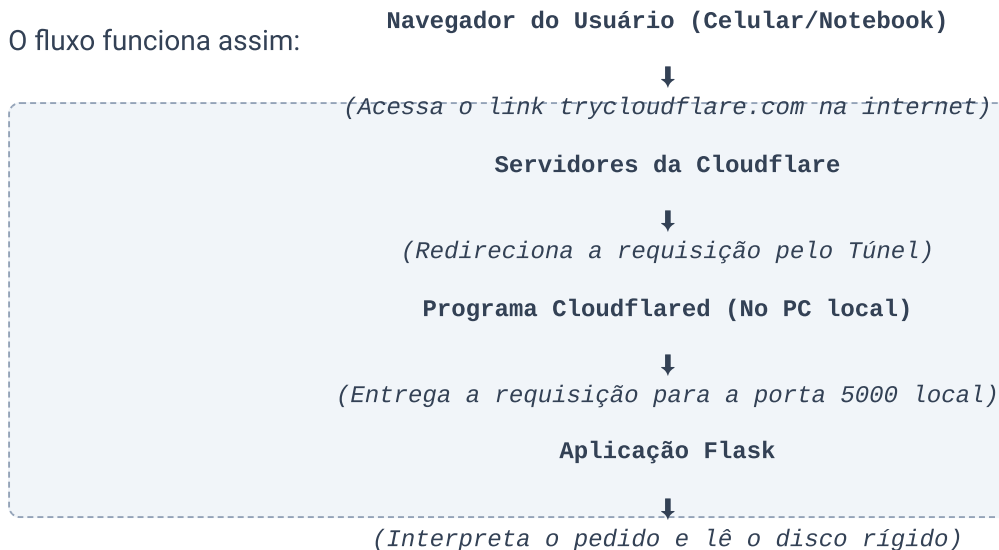
Continuando com a nossa lista de ferramentas do projeto:

- **Cloudflare Tunnel:** É um serviço de rede da empresa Cloudflare. Ele cria um "túnel" seguro entre o seu computador local e a internet. Serve para expor um site que está rodando apenas na sua máquina para o mundo todo, sem precisar abrir portas no seu roteador (o que seria perigoso e trabalhoso).
- **Cloudflared:** É o programa (aplicativo) que instalamos no computador para fazer o Cloudflare Tunnel funcionar. O script PowerShell avisa ao *cloudflared* para criar o túnel para o Flask.
- **SMTP (Simple Mail Transfer Protocol):** É a regra padrão (protocolo) da internet para envio de e-mails. Usamos a biblioteca de SMTP nativa do Python para fazer nossa aplicação conversar com o servidor de e-mail.
- **Gmail:** O serviço de e-mail do Google. No nosso projeto, usamos uma conta do Gmail como remetente. O Python faz login nessa conta e dispara o link do Cloudflare para o seu e-mail pessoal.
- **NSSM (Non-Sucking Service Manager):** É uma ferramenta que transforma programas comuns em "Serviços do Windows". Ele pega o nosso script PowerShell e o empacota para que o Windows o inicie nos bastidores logo que o computador é ligado, antes mesmo de você digitar sua senha na tela de bloqueio.
- **Windows Services (Serviços do Windows):** É a parte do sistema operacional que roda programas em segundo plano. Relacionando tudo: O NSSM cria um Serviço do Windows que roda o PowerShell, que por sua vez roda o Flask e o Cloudflare.

O fluxo de trabalho foi pensado para que cada ferramenta faça apenas aquilo em que é melhor, criando uma linha de montagem automatizada.

3. Arquitetura do Projeto

Entender a arquitetura é entender como as peças conversam entre si. Existe uma grande diferença entre o computador (onde os arquivos estão fisicamente), a aplicação Flask (que lê os arquivos e cria o site) e o navegador (onde você vê o site).



Vamos detalhar o caminho de uma requisição: Imagine que você quer baixar uma foto. Você abre o link no seu celular. Esse celular não fala diretamente com o seu computador. Ele manda o pedido para a internet, que chega na Cloudflare.

A Cloudflare sabe que aquele link pertence ao túnel aberto pelo `cloudflared` no seu PC. Ela envia o pedido por esse túnel. O `cloudflared` recebe o pedido e o entrega na porta da sua casa (porta 5000), onde a aplicação Flask está escutando.

O Flask entende: "Ele quer a foto de férias". O Flask vai até o sistema de arquivos do computador, pega a foto, e faz o caminho inverso, devolvendo a imagem pelo túnel até chegar na tela do seu celular.

4. Aplicação Flask

O arquivo principal do código Python é o `app.py`. Ele é o cérebro da nossa interface web. O Flask cria um **servidor local**, que significa que ele atende pedidos web dentro do próprio computador. Ele roda em um endereço interno chamado **localhost** (que significa "esta mesma máquina") na **porta 5000** (pense na porta como um guichê de atendimento específico).

Dentro do `app.py`, nós criamos **rotas**. Uma rota é uma regra do tipo: "Se alguém acessar o endereço /download, execute a função de baixar arquivo". Quando o usuário acessa uma rota, ele faz uma **requisição HTTP** (um pedido de dados). O Flask processa, busca o arquivo e devolve uma **resposta HTTP**.

Para a página ficar bonita, o Flask usa **templates** (arquivos HTML que funcionam como moldes, onde o Python preenche o nome dos arquivos) e **arquivos estáticos** (imagens de ícones e o arquivo CSS com as cores).

5. Sistema de Arquivos

A grande missão do Flask aqui não é mostrar um blog, mas sim exibir o que tem no seu HD. Para isso, usamos bibliotecas do Python que "conversam" com o sistema de arquivos do Windows.

Quando você acessa a página inicial, o Python lê tudo o que está dentro da pasta que definimos como base e envia para a tela do navegador. Nós programamos funcionalidades como:

- **Navegar:** Entrar em diretórios clicando neles e voltar para os anteriores.
- **Visualizar e Baixar:** Clicar em um arquivo para fazer download.
- **Upload:** Um botão na tela onde você envia um arquivo do celular, e o Flask o salva no computador.
- **Gerenciar:** Botões para criar novas pastas.

Mas, para fazer isso, o controle dos caminhos (onde o Python pode ou não pode entrar no computador) é absolutamente crítico, o que nos leva ao próximo assunto.

6. Segurança dos Caminhos

Imagine que nossa pasta de arquivos permitida se chama `C:\MeusArquivos`. Quando o usuário pede para ver uma subpasta, o navegador envia o nome dela. Mas e se um usuário mal-intencionado enviar um caminho modificado, como `../../../../Windows/System32`?

Isso é conhecido como **Directory Traversal** (ou Path Traversal). Se simplesmente aceitarmos qualquer caminho, o atacante consegue "voltar" pastas (usando o `../`) e acessar arquivos sensíveis do sistema operacional, fora da nossa pasta permitida.

Para evitar isso, criamos uma função de validação (por exemplo, `caminho_seguro`). O que ela faz é pegar o caminho que o usuário pediu, resolver todos os "voltar pasta" e calcular o caminho absoluto real (usando `realpath` no Python). Depois, ela verifica matematicamente: esse caminho final ainda começa com `C:\MeusArquivos`? Se sim, o acesso é liberado. Se não, o acesso é bloqueado. Essa trava é simples, mas essencial para que o projeto não exponha o seu computador inteiro. Contudo, é importante lembrar que isso sozinho não torna o sistema imune a todas as falhas possíveis na internet.

7. Cloudflare Tunnel (Parte 1)

Como vimos, o Flask roda em `http://localhost:5000`. O problema é que `localhost` (ou um endereço IP local como `192.168.1.5`) só existe dentro da sua casa, no seu Wi-Fi. A internet externa não faz ideia de como encontrar esse endereço, porque ele fica atrás da proteção do seu roteador.

No passado, para acessar isso da rua, você teria que abrir portas no roteador, configurar IP dinâmico e expor seu computador a ataques. O **Cloudflare Tunnel** muda esse jogo.

Em vez de você abrir uma porta para a internet entrar, o programa `cloudflared` no seu computador faz uma conexão de dentro para fora até os servidores da Cloudflare. É como cavar um túnel seguro. Como a conexão saiu de dentro do seu computador, o roteador permite.

O comando que utilizamos no terminal é bem parecido com este:

```
cloudflared tunnel --url http://localhost:5000
```

Ele significa: "Cloudflared, crie um túnel e tudo o que chegar nele, entregue no endereço local `http://localhost:5000`". Neste formato rápido que usamos, ele gera um endereço temporário (terminado em `trycloudflare.com`). É muito importante entender que esse link é descartável; se o computador reiniciar, um link completamente diferente será gerado.

8. Script `trycloudflared.ps1`

Como o link muda toda vez, seria inviável você ter que abrir o terminal, rodar o Flask, rodar o túnel e copiar o link manualmente todo dia. Por isso, criamos a automação principal: o `trycloudflared.ps1`.

Sequência de Automação (PowerShell)

1. Apaga antigo `link.txt`
↓
2. Inicia o Flask (`app.py`) em segundo plano
↓
3. Pausa de alguns segundos (Espera o Flask carregar)
↓
4. Inicia o Cloudflared
↓
5. Lê a saída, encontra a URL e salva em `link.txt`
↓
6. Executa `enviar_email.py` lendo o `link.txt`

As pausas (esperas) no script são cruciais. Se o script tentar abrir o túnel antes do Flask estar pronto, dá erro. Se tentar enviar o e-mail antes do túnel gerar a URL, manda um e-mail vazio. O PowerShell coordena essa dança com o tempo certo para cada etapa.

9. Script enviar_email.py

A última etapa da nossa automação é o arquivo `enviar_email.py`. Quando o PowerShell finalmente consegue o link do dia, ele "chama" esse script em Python passando o link gerado junto com o comando. Fica mais ou menos assim:

```
python enviar_email.py "https://exemplo.trycloudflare.com"
```

O papel desse script é puramente logístico. Ele pega o link e cria um e-mail com o assunto "Servidor Online - Link de Acesso". Usando o protocolo SMTP, o script faz o login em uma conta do Gmail e envia a mensagem para o seu e-mail pessoal. Para fazer o login funcionar no Gmail hoje em dia, precisamos gerar uma "Senha de Aplicativo" na conta do Google, que funciona como uma senha especial só para scripts.

10. Arquivo .env

Colocar o seu e-mail e a sua senha de aplicativo diretamente no código do `enviar_email.py` é uma péssima ideia. Se você mostrar esse código para alguém ou salvar no GitHub, todos saberão a sua senha.

Para resolver isso, usamos um arquivo chamado `.env`. É um simples arquivo de texto que guarda configurações sensíveis. Um exemplo fictício seria:

```
GMAIL_USUARIO=servidor.exemplo@gmail.com GMAIL_SENHA_APP=abcd_efgh_ijkl_mnop  
EMAIL_DESTINO=usuario.exemplo@gmail.com
```

No código Python, usamos uma biblioteca chamada `python-dotenv` para ler essas informações de forma invisível. E a regra de ouro: o arquivo `.env` é adicionado ao `.gitignore`, garantindo que ele jamais seja publicado na internet, protegendo suas credenciais de forma segura.

11. NSSM (Non-Sucking Service Manager)

Até o momento, temos um script PowerShell perfeito. Mas ele tem um problema: para funcionar, você precisa ligar o computador, digitar a senha do Windows, abrir o PowerShell e dar o play. Se o computador reiniciar após uma queda de energia, o servidor não volta sozinho.

É aqui que entra o **NSSM**, cujo nome peculiar em inglês brinca que ele é um "gerenciador de serviços que não é ruim". Sua função é pegar um aplicativo comum e transformá-lo em um serviço embutido do Windows. O NSSM **não** inicia o Flask diretamente; ele encapsula o nosso script `trycloudflared.ps1`.

Estrutura de execução: Windows → Serviço (NSSM) → PowerShell → `trycloudflared.ps1` → (Flask + Cloudflare + E-mail).

Essa magia permite que o projeto seja iniciado automaticamente pelo sistema operacional na tela de carregamento, mesmo sem nenhum usuário fazer login. Com o serviço instalado, você gerencia tudo pelo próprio Windows:

- **Instalar:** `nssm install servidorpython "C:\Caminho\Para\PowerShell.exe"`
- **Iniciar o servidor:** `nssm start servidorpython`
- **Parar o servidor:** `nssm stop servidorpython`
- **Reiniciar:** `nssm restart servidorpython`
- **Remover:** `nssm remove servidorpython`

12. Inicialização Completa do Sistema

Com tudo montado, o que acontece de verdade quando você aperta o botão de ligar o computador?

1. O Windows inicia seus processos básicos.
2. O serviço configurado pelo NSSM entra em ação automaticamente.
3. O NSSM abre o PowerShell silenciosamente.
4. O PowerShell executa o `trycloudflared.ps1`.
5. O script inicia o servidor Flask, que começa a escutar na porta 5000.
6. Logo depois, o script inicia o Cloudflared.
7. O Cloudflare gera o endereço longo temporário e avisa o terminal.
8. O PowerShell captura esse endereço e salva em `link.txt`.
9. Passado o intervalo de segurança configurado, o `enviar_email.py` entra em cena.
10. O Python envia o e-mail via Gmail com o link recém-criado.
11. O processo termina. O usuário recebe a notificação no celular e já pode acessar os arquivos.

13. Exemplo de Uso Prático

Vamos imaginar a rotina de um estudante. Ele tem o computador servidor em casa. Às 7h da manhã, antes de sair para o trabalho, ele apenas aperta o botão de ligar do PC e sai de casa. Ele nem precisa digitar a senha no Windows.

No ônibus, ele recebe uma notificação no celular: um e-mail automático chegou com um link terminado em `trycloudflare.com`. Quando ele chega no escritório e precisa de um documento que estava no HD da casa dele, ele abre aquele link no navegador do trabalho. A interface web amigável do Flask aparece. Ele navega pelas pastas, encontra o documento em PDF e clica em download. Tudo isso rodando de forma invisível no quarto dele.

14. Vantagens da Solução

Este projeto apresenta várias vantagens interessantes, principalmente para quem deseja entender como o armazenamento em nuvem funciona por trás dos panos:

- **Baixo custo:** Utiliza hardware que você já possui em casa e ferramentas gratuitas.
- **Armazenamento local (Controle):** Os arquivos não estão em servidores de empresas terceiras, mas fisicamente sob a sua supervisão.
- **Acesso universal:** Acessível por qualquer navegador moderno, sem precisar instalar aplicativos no celular.
- **Automação inteligente:** Uma vez configurado, o sistema roda sozinho sem intervenção humana.
- **Curva de aprendizado:** É um projeto excelente que integra desenvolvimento web, automação, redes, serviços externos e administração do Windows de uma só vez.

15. Limitações do Projeto

É importante manter os pés no chão. Esta solução acadêmica e prática tem limites claros e não substitui de imediato um serviço profissional de nuvem gigante:

- O seu computador servidor precisa estar fisicamente ligado e consumindo energia.
- Depende da sua conexão de internet local. Se a sua casa ficar sem internet ou faltar energia, o acesso externo cai.
- O link do trycloudflare.com muda toda vez. Você não tem um site com nome fixo.
- O desempenho do upload e download depende fortemente do seu pacote de internet banda larga (velocidade de upload residencial).
- O projeto básico não possui um sistema robusto de login de usuários, dependendo do link longo como fator principal de segurança momentânea.

16. Segurança e Boas Práticas

No desenvolvimento desse servidor, precisamos adotar medidas sérias para evitar problemas:

- Nunca publique o arquivo `.env` ou senhas diretamente no código em locais públicos como o GitHub. Use sempre o `.gitignore`.
- Trate a senha de aplicativo do Gmail com rigor. Se ela vazar, a pessoa pode usar o seu e-mail para enviar spam.
- Valide com precisão os caminhos dos diretórios para evitar o vazamento de arquivos do sistema operacional (Directory Traversal).
- Controle severamente quem recebe o link do Cloudflare. Neste modelo sem tela de login, qualquer pessoa com o link pode ver e baixar os arquivos.
- Mantenha as bibliotecas do Python e o sistema Windows sempre atualizados.
- Não armazene informações extremamente sensíveis ou dados financeiros em um projeto de testes sem criptografia.

17. Estrutura do Projeto

A organização dos arquivos no computador seguiu uma padronização lógica para separar código web, visualização e automação:

```
Servidor_de_Armazenamento/ | ├── app.py (Aplicação Flask) |── .env (Configurações de e-mail e senhas secretas) |── .gitignore (Diz ao Git para ignorar o .env) |── pastas_arquivos/ (Local onde ficam os arquivos de fato) |── templates/ |   |── index.html (Página HTML que o usuário vê) |   |── static/ |   |   |── style.css (Cores e estilo visual da página) |   |   |── scripts/ |   |   |   |── trycloudflared.ps1 (Automação do Windows) |   |   |   |── enviar_email.py (Disparo da mensagem com o link)
```

18. Problemas Encontrados no Desenvolvimento

Todo projeto passa por desafios. Durante a criação, um dos maiores problemas foi a sincronia de tempo na automação. Inicialmente, o script tentava abrir o túnel enquanto o Flask ainda estava iniciando, o que causava travamentos. Depois, percebemos que o `cloudflared` demora alguns segundos para negociar o link com os servidores; se o script Python tentasse enviar o e-mail imediatamente, o e-mail chegava sem link nenhum.

Outro ponto chato foi a autenticação SMTP. O Google bloqueou o envio nas primeiras tentativas alegando ser um "aplicativo menos seguro", o que nos forçou a pesquisar e implementar a "Senha de Aplicativo" específica para o código. Esses testes foram fundamentais para criarmos as "esperas" (pausas no código) da automação final.

19. Possíveis Melhorias Futuras

Este sistema pode evoluir muito. Algumas possibilidades que poderiam ser implementadas no futuro são:

- **Autenticação Real:** Uma tela de login exigindo usuário e senha antes de mostrar as pastas.
- **Domínio Próprio e HTTPS permanente:** Comprar um site fixo e usar a versão completa da Cloudflare em vez do link temporário.
- **Limites de Espaço:** Controlar quanto o usuário pode enviar (upload) e reservar cotas no disco rígido.
- **Painel Administrativo:** Uma aba no site onde o dono possa ver logs e monitorar quem baixou qual arquivo.
- **Banco de Dados:** Passar a guardar as informações dos usuários e histórico de atividades em um banco estruturado (como SQLite ou PostgreSQL).

20. Conclusão

Este trabalho mostra que criar uma solução em nuvem particular é totalmente viável. O projeto vai muito além de um simples servidor Flask. Ele integrou ferramentas de rede externas (Cloudflare), manipulação de dados locais, disparo de e-mails via SMTP e controle profundo dos serviços do sistema operacional Windows (PowerShell e NSSM).

Desenvolver esta arquitetura demonstrou que pequenos pedaços de tecnologia — desde um arquivo HTML até um gerenciador de serviços complexo — podem conversar entre si de forma harmoniosa, transformando uma máquina parada no canto da sala em uma verdadeira ferramenta automatizada de produtividade.